

Analyzing Task Completion Patterns in an Early-Stage Startup

A Machine Learning Approach to Predicting Task Delays

Author	Bedirhan Ceylan
Course	DSA 210 — Introduction to Data Science (Spring 2026)
Section	A — Teaching Assistant: Erfan Tarmohammadi
Repository	github.com/bedirhancceylan/DSA210-Project

Abstract

This report investigates which factors are associated with task delays in the development workflow of Sollen, an early-stage startup operating in the occupational health and safety domain. Using a dataset of 486 task records exported from Azure DevOps and spanning 18 months of active development (October 2024 – April 2026), we apply exploratory data analysis, three non-parametric hypothesis tests, and five supervised classification models to identify the drivers of completion delays. The chi-square test reveals that task category alone is not a statistically significant predictor of delay status ($p = 0.275$), motivating a feature-engineered machine learning approach. After constructing ten leakage-free features — including developer workload at task creation, team-wide concurrent load, and historical developer performance — Random Forest emerges as the strongest model with a Test ROC-AUC of 0.741 and stable cross-validation performance (0.708 ± 0.040). Permutation importance analysis indicates that developer-history features and team load contribute more predictive signal than task category, suggesting that task-level interventions should focus on workload management rather than category-specific process changes.

1. Motivation

Early-stage startups operate under uncertainty: small teams, shifting priorities, and limited tooling for tracking why work takes longer than expected. As a Founding Chief of Staff at Sollen — an early-stage startup currently in the product development phase, building a B2B SaaS and hardware platform for occupational health and safety management — I observed task slippage patterns that were costly but poorly understood. Some categories of work consistently ran late, certain developers seemed to absorb delays at higher rates, and intuitively the team-wide load at the moment a task was created appeared to influence its eventual completion time. None of these intuitions had been formally tested.

This project applies the data science pipeline taught in DSA 210 — exploratory analysis, hypothesis testing, feature engineering, and supervised machine learning — to a real operational dataset I have access to as part of my role. The dual goal is academic (demonstrating end-to-end competence with the methods covered in the course) and practical (producing an interpretable model that surfaces actionable signals for the team).

Three concrete questions structure the investigation:

- Is task duration meaningfully different across departments (tech, design, research, product, operations)?
- Is task category — by itself — sufficient to predict whether a task will be delayed?
- Can a feature-engineered machine learning model identify which conditions at task creation are most predictive of eventual delay?

2. Data Source

2.1 Origin and Collection

All data was exported from the Azure DevOps work item history of Sollen's primary product repository. The export covers every task that was created and subsequently closed between 18 October 2024 and 7 April 2026 — a continuous 18-month window of active development. Tasks that were still open at the time of extraction were excluded, since they have no completion date and therefore cannot be classified as on-time or delayed. Cancelled or duplicate items were also removed. The resulting dataset contains 486 closed tasks.

Personal identifiers were removed before analysis: developer names were replaced with anonymous labels (Developer_A through Developer_N, plus a placeholder Unassigned). Free-text task titles were retained but are not used as direct features in the model — only their length (word count) is engineered as a complexity proxy. The repository owner approved the use of this data for academic purposes prior to extraction.

2.2 Schema

Column	Type	Description
task_id	int	Unique Azure DevOps work item ID

title	string	Free-text task title (used only for word count)
task_category	string	Department: tech, design, research, product, operations
priority_level	string	low, medium, high (severely imbalanced — see §3.2)
assigned_to	string	Anonymized developer label (15 distinct values)
creation_date	datetime	Timestamp when the task was opened
completion_date	datetime	Timestamp when the task was marked complete
actual_duration_days	float	Days between creation and completion
delay_label	string	delayed if duration > 1.5× category median, else on_time
has_urgent_tag	int	Binary flag (severely imbalanced — see §3.2)

2.3 Target Variable Construction

The `delay_label` was constructed from `actual_duration_days` using a category-relative threshold: a task is labelled delayed if its actual duration exceeds 1.5× the median duration for its category. This category-relative definition was preferred over an absolute threshold (e.g. any task taking more than 30 days) because the typical duration of a task varies meaningfully by department — a 14-day design task may be normal, while a 14-day research task is unusually long. Using the median rather than the mean makes the threshold robust to extreme outliers, which are present in the data (some tasks ran for over 300 days).

The resulting class balance is 169 delayed (34.8%) and 317 on-time (65.2%) — moderately imbalanced but suitable for stratified sampling without aggressive resampling techniques.

3. Data Analysis

3.1 Exploratory Data Analysis

EDA was performed in `eda.ipynb` and produced 11 figures stored under the `plots/` directory. The dataset contains no missing values across any of the 10 raw columns. Tasks are heavily concentrated in the tech department (208 tasks, 42.8% of all work), followed by research (91), design (83), product (70), and operations (34). Across 15 distinct developers, workload is highly uneven: the top three developers (A, F,

D) collectively account for 246 tasks (50.6%), while the bottom three (B, K, L) account for fewer than 15 tasks combined.

Task durations are heavily right-skewed. The mean duration is 32.7 days but the median is 20.7 days, and the distribution has a long tail extending to 343 days. This skew motivated the use of non-parametric tests in the hypothesis testing phase and a log-transformation for visualization purposes. Duration also varies by category in ways that match operational intuition: product tasks have the longest median duration (32.1 days), while operations and research tasks have the shortest (9.1 and 10.0 days respectively).

3.2 Feature Quality Issues Discovered During EDA

Two columns turned out to carry essentially no information and were dropped before the modeling phase:

- `priority_level` — 482 of 486 tasks (99.2%) are tagged 'medium', with only 3 'high' and 1 'low'. This reflects a workflow where the team rarely uses the priority field rather than a true distribution of urgency.
- `has_urgent_tag` — 483 of 486 tasks (99.4%) have value 0. Same root cause as above.

These would inflate the feature count without contributing predictive signal and were explicitly removed during feature engineering (see §3.4).

3.3 Hypothesis Testing

Three hypothesis tests were conducted, each chosen for a specific structural reason in the data.

3.3.1 Test 1 — Kruskal-Wallis

Question: Does task duration differ across the five categories?

H_0 : Duration distributions are identical across categories.

H_1 : At least one category has a different duration distribution.

The Kruskal-Wallis H test was chosen instead of one-way ANOVA because the duration variable is heavily right-skewed and ANOVA's normality assumption does not hold. Kruskal-Wallis is the non-parametric alternative and operates on rank order rather than raw values.

Result: $H = 39.97$, $p < 0.0001 \rightarrow$ reject H_0 .

Conclusion: there is a statistically significant difference in task duration across categories. Inspection of the per-group medians confirms that product (32.1 days) and tech (28.9 days) tasks take substantially longer than operations (9.1 days) and research (10.0 days).

3.3.2 Test 2 — Chi-Square Test of Independence

Question: Is task category associated with delay status?

H_0 : Task category and delay status are independent.

H_1 : Task category and delay status are associated.

Result: $\chi^2 = 5.13$, $df = 4$, $p = 0.275 \rightarrow$ fail to reject H_0 .

Conclusion: there is no statistically significant association between task category and delay status at $\alpha = 0.05$. This is the most analytically important result in the study, even though it is a negative result. Combined with Test 1, it tells us: categories differ in absolute duration, but they do not differ in their relative tendency to be delayed (which makes sense — the delay label is computed within each category). The implication for the modeling phase is that category alone is a weak feature, and any predictive signal must come from feature interactions or other variables. This directly motivated the inclusion of workload and developer-history features in the ML pipeline.

3.3.3 Test 3 — Mann-Whitney U Test

Question: Do delayed tasks have significantly longer durations than on-time tasks?

H_0 : Duration distributions are identical between delayed and on-time tasks.

H_1 : Delayed tasks have larger durations (one-sided alternative).

Result: $U = 50,252$, $p < 0.0001$, rank-biserial $r = 0.876 \rightarrow$ reject H_0 .

Conclusion: this test confirms that delayed tasks have significantly longer durations than on-time tasks, with a very large effect size. This result is partially trivial by construction — the delay label is itself derived from duration — but the test was included to formally quantify the magnitude of the difference and verify that the labelling threshold ($1.5 \times$ median) produces a meaningful separation rather than an arbitrary cut.

3.4 Feature Engineering

The feature engineering phase (ml.ipynb) constructed a leakage-free feature matrix from the raw dataset. The most consequential design decision was to identify and remove all features that could leak information about the target. Three columns were dropped for direct leakage: `actual_duration_days`, `completion_date`, and `delay_label` itself were all known only after the task was completed and would give the model perfect knowledge of the answer. Two additional columns (`priority_level`, `has_urgent_tag`) were dropped for zero variance (see §3.2), and `task_id` was dropped as a non-feature identifier.

Ten features were constructed:

Feature	Type	Description
<code>task_category</code>	categorical	Direct from raw column
<code>assigned_to</code>	categorical	Direct from raw column (anonymized)
<code>creation_day</code>	categorical	Day of week task was created (Monday–Sunday)
<code>creation_month_num</code>	numeric	Month of creation (1–12)
<code>is_weekend_creation</code>	binary	Whether the task was created on

		Saturday or Sunday
title_word_count	numeric	Word count of the task title (complexity proxy)
team_total_load_at_creation	numeric	Number of open tasks across the team at the moment this task was created
developer_workload_at_creation	numeric	Number of open tasks for the assigned developer at creation time
dev_historical_delay_rate	numeric	Developer's historical delay rate using only tasks created before this one
dev_avg_past_duration	numeric	Developer's historical mean duration using only past tasks

The two historical features (`dev_historical_delay_rate` and `dev_avg_past_duration`) were computed with strict temporal ordering: for each task, only tasks with strictly earlier `creation_date` were used to compute the developer's prior performance. This prevents future leakage that would otherwise occur if the entire history of a developer's work were used to predict an early task. For a developer's first task, no prior history exists, so these features were filled with the global mean — a cold-start strategy that affects 15 rows total (one per developer).

3.5 Modeling Pipeline

All five models were trained inside scikit-learn Pipeline objects that combine preprocessing and the classifier into a single object. Categorical features (`task_category`, `assigned_to`, `creation_day`) are one-hot encoded; numeric features are standard-scaled. Wrapping preprocessing inside the pipeline ensures that during cross-validation, the encoder and scaler are fit only on the training fold and applied to the test fold — preventing data leakage from the test fold into the preprocessing step. After one-hot encoding, the feature space expands from 10 to 34 columns.

Two evaluation protocols were used in parallel for each model:

- Stratified 5-fold cross-validation with `shuffle=True` and `random_state=42`, reporting accuracy, precision, recall, F1, and ROC-AUC as mean $\pm$ standard deviation across the five folds.
- A single 80/20 stratified train/test split (`random_state=42`) for confusion matrices, ROC curves, and precision-recall curves on a held-out test set of 98 samples.

Five models were compared:

- Logistic Regression — `class_weight='balanced'` to handle the 65/35 imbalance, `max_iter=1000`.
- Decision Tree — `max_depth=6`, `class_weight='balanced'`. Depth was capped to prevent overfitting on the relatively small dataset of 486 rows.
- k-Nearest Neighbors — `k=5`, no class weighting (kNN does not accept it directly).

- Random Forest — 200 trees, max_depth=8, class_weight='balanced', n_jobs=-1.
- Gradient Boosting — 200 boosting stages, learning_rate=0.05, max_depth=3. Note: scikit-learn's GradientBoostingClassifier does not accept class_weight; the imbalance is handled implicitly through the loss function and the depth/learning-rate tuning.

4. Findings

4.1 Model Comparison

All five models were evaluated on the same data with the same protocol. Results are summarized below. The ROC-AUC metric is the primary criterion because it is threshold-independent and not biased by class imbalance the way raw accuracy can be.

Model	CV ROC-AUC	Test ROC-AUC	Test Accuracy	Test F1 (delayed)
Logistic Regression	0.577 ± 0.039	0.639	0.62	0.53
Decision Tree	0.582 ± 0.030	0.573	0.60	0.47
kNN (k=5)	0.662 ± 0.050	0.710	0.70	0.47
Gradient Boosting	0.684 ± 0.033	0.725	0.71	0.48
Random Forest	0.708 ± 0.040	0.741	0.71	0.55

Random Forest is the best model on every metric reported. Three observations are worth noting:

- The two linear/single-tree baselines (Logistic Regression and Decision Tree) perform poorly — barely above the random baseline (ROC-AUC 0.5). This confirms the chi-square finding from §3.3.2: the predictive signal is not in any single feature but in the interactions between features, which tree ensembles are better at capturing.
- The Gradient Boosting model is competitive (Test ROC-AUC 0.725) but does not beat Random Forest. On a small dataset of 486 rows, the regularization effect of Random Forest's bootstrap aggregation appears to outperform Gradient Boosting's sequential refinement.
- Random Forest's CV standard deviation (± 0.040) is small relative to its mean (0.708), indicating that performance is stable across different data splits — the model is not overfitting to any particular fold.

4.2 Per-Fold Stability Check (Random Forest)

To verify stability, the per-fold metrics for Random Forest are reported separately. ROC-AUC stays in the range 0.65–0.76 across all five folds, which is consistent enough to trust the mean as a real performance estimate.

Fold	Accuracy	Precision	Recall	F1	ROC-AUC
Fold 1	0.673	0.531	0.500	0.515	0.755
Fold 2	0.691	0.538	0.636	0.583	0.747
Fold 3	0.680	0.552	0.471	0.508	0.692
Fold 4	0.680	0.565	0.382	0.456	0.698
Fold 5	0.660	0.514	0.529	0.522	0.646
Mean	0.677	0.540	0.504	0.517	0.708

Std	0.011	0.019	0.092	0.045	0.045

4.3 Feature Importance

Two complementary measures of feature importance were computed. Impurity-based importance is available directly from the fitted Random Forest model but can be biased toward high-cardinality categorical features (such as `assigned_to` with 15 levels). Permutation importance is a more reliable measure: it directly quantifies how much the model's ROC-AUC drops when a single feature is randomly shuffled on the test set. Permutation importance was computed with 30 repeats and is reported as the primary ranking.

Feature	Importance (mean ROC-AUC drop)	Std
<code>creation_month_num</code>	0.063	0.023
<code>task_category</code>	0.043	0.015
<code>dev_historical_delay_rate</code>	0.039	0.011
<code>assigned_to</code>	0.039	0.018
<code>team_total_load_at_creation</code>	0.033	0.016
<code>dev_avg_past_duration</code>	0.027	0.009
<code>developer_workload_at_creation</code>	0.007	0.011
<code>is_weekend_creation</code>	0.003	0.003
<code>title_word_count</code>	-0.010	0.013
<code>creation_day</code>	-0.029	0.029

Key observations from the ranking:

- `creation_month_num` is the strongest single predictor — confirming that the team's delay rate varies meaningfully across the year, likely tracking external factors such as funding cycles, investor pitches, or product releases that change task urgency.
- Although the chi-square test in §3.3.2 found that `task_category` alone was not significantly associated with delay, it does contribute predictive power once combined with other features in a non-linear model. The two findings are not contradictory — chi-square measures marginal association, permutation importance measures conditional contribution given everything else.
- Developer-history features (`dev_historical_delay_rate`, `dev_avg_past_duration`) and team load are non-trivial contributors. The combination of these three features carries comparable weight to `task_category` itself, validating the design decision to engineer them.
- `creation_day`, `title_word_count`, and `is_weekend_creation` contribute nothing meaningful and would be candidates for removal in a leaner production model.

4.4 Confusion Matrix Interpretation

On the held-out test set of 98 samples (64 on-time, 34 delayed), Random Forest achieves the following:

- True positives (correctly predicted delays): 17
- True negatives (correctly predicted on-time): 53
- False positives (predicted delay, was on-time): 11
- False negatives (predicted on-time, was delayed): 17

The model is more conservative than aggressive: it has higher specificity ($53/64 = 0.83$) than sensitivity ($17/34 = 0.50$) for the delayed class. In a hypothetical operational deployment, this profile would mean the model misses about half of the actual delays but rarely raises false alarms. For a planning tool intended to surface tasks that need attention, the better operating point would likely be a lower decision threshold than the default 0.5 — trading some false alarms for higher recall on actual delays.

5. Limitations and Future Work

5.1 Limitations

Several limitations of this study should be noted:

- **Sample size.** The dataset contains 486 tasks. While sufficient for stratified cross-validation and the methods used here, this is a small sample for machine learning. The wider confidence intervals on per-fold metrics (especially F1, where std is 0.045 on a mean of 0.517) reflect this. Doubling the dataset would meaningfully tighten these estimates.
- **Single-team scope.** All 486 tasks come from one team in one company. Patterns observed here — such as the strong importance of `creation_month_num` — may reflect that specific team's operating rhythm and would not necessarily generalize to other startups or larger organizations.
- **Definition of 'delayed'.** The $1.5\times$ category-median threshold is a conventional choice but is not validated against any external definition of business-meaningful delay. A real product deployment would benefit from a domain-derived threshold (e.g. tasks that missed a sprint boundary or blocked a downstream dependency).
- **Missing context features.** Several factors that would intuitively influence task duration are not captured in the Azure DevOps export: task complexity beyond word count (e.g. story points), the number of dependencies, whether the developer was simultaneously on vacation or other commitments, or external blockers (waiting on a third-party). Including these would likely raise the ceiling of model performance.
- **Two zero-variance features** (`priority_level`, `has_urgent_tag`). These were dropped for analytical reasons but their absence is also a finding: the team is not actively using these workflow fields, which is itself a process-level observation.
- **Model interpretability vs. accuracy trade-off.** Random Forest is more interpretable than a neural network but less interpretable than a single decision tree or logistic regression. For deployment as a planning tool, a slightly worse but more transparent model (e.g. a tuned logistic regression with engineered interaction features) might be preferable to the team.

5.2 Future Work

Several extensions would meaningfully improve the system:

- **Threshold tuning.** As noted in §4.4, the default classification threshold of 0.5 produces a conservative model. A precision-recall curve analysis with a domain-relevant cost matrix would identify the operating point that is most useful for the team.
- **Hyperparameter tuning with GridSearchCV.** The current model uses a reasonable but unoptimized set of hyperparameters. A grid search over `n_estimators`, `max_depth`, and `min_samples_leaf` would likely yield a small but real improvement in CV ROC-AUC.
- **Time-series cross-validation.** The current 5-fold CV randomly shuffles the data. Since the dataset spans 18 months and the team's behavior may have evolved, a more honest evaluation would use a `TimeSeriesSplit` that always trains on past data and tests on future data.

- Richer feature set. Pulling in additional Azure DevOps fields (story points, parent/child task relationships, sprint identifiers) and external context (national holidays, team velocity rolling averages) would expand the feature space.
- Multi-target prediction. The current model predicts a binary delayed/on-time label. Extending to a regression that predicts the actual duration would be more informative for planning, and could be done with the same engineered features.
- Deployment. The persisted model (`final_model_random_forest.pkl`) is reproducible but not integrated into the team's tooling. A natural next step is a lightweight web interface where a developer can enter a new task's category, expected assignee, and current team load, and receive a delay probability.
- Comparison with simpler heuristics. Before any model is deployed, it should be benchmarked against human intuition (asking the team lead to predict each test-set outcome) and trivial baselines (predicting based only on the assigned developer's prior delay rate). If the gap is small, a model is not worth the operational complexity.

6. Conclusion

This study applied the full data science pipeline to a real operational dataset of 486 task records from an early-stage startup. The key analytical finding came from the chi-square test, which showed that task category alone is not associated with delay status — a negative result that motivated a more sophisticated feature engineering approach. The resulting Random Forest model achieves a Test ROC-AUC of 0.741, with developer history, team workload, and timing features all contributing meaningful predictive signal.

From an operational perspective, the most actionable insight is that delay is more strongly predicted by the conditions of work (who is doing it, how loaded they are, what month it is) than by the nature of the work itself. This suggests that interventions aimed at reducing delay should focus on workload smoothing and developer assignment rather than on category-specific process changes.

7. Reproducibility

All code, data, and figures are available in the project repository. The notebooks should be run in order for full reproducibility:

```
git clone https://github.com/bedirhancceylan/DSA210-Project.git
cd DSA210-Project
python3 -m venv venv
source venv/bin/activate          # macOS/Linux
pip install -r requirements.txt
jupyter notebook
```

Notebook execution order: `eda.ipynb` → `ml.ipynb` → `ml_day2.ipynb` → `ml_day3.ipynb` → `ml_day4.ipynb`. All notebooks use `random_state=42` for any stochastic operation, so results are exactly reproducible.

The Git repository is tagged at two milestones for grading reference:

- milestone1 (cc73f8a) — EDA + hypothesis testing + project proposal (April 14)
- milestone2 (722b23b) — Complete ML pipeline + final Random Forest model (April 29)

8. AI Usage Disclosure

Per the DSA 210 project guidelines, all use of AI tools must be explicitly documented. This section describes how Claude (Anthropic) was used during the project and what was contributed by the author.

8.1 What Claude Was Used For

- Iterative discussion of feature engineering decisions, particularly the leakage-prevention logic for historical developer features (computing past-only statistics).
- Generation of notebook scaffolding (code structure, imports, plot styling, and consistent naming conventions across the four ML notebooks).
- Suggestions for which hypothesis tests fit the data structure (Kruskal-Wallis for non-parametric multi-group comparison; chi-square for categorical association; Mann-Whitney U for one-sided two-group comparison on skewed data).
- Drafting and revising this final report and the project README.
- Git workflow guidance for the milestone tag and clean commit history.

8.2 What the Author Contributed

- Project topic, scope, and access to the source data.
- All decisions about which results to report and how to interpret them.
- All claims and conclusions in this report.
- Local execution, debugging, and verification of every notebook cell.
- All analytical judgments — including the decision to drop `priority_level`, the choice of the 1.5× median threshold for delay labelling, and the interpretation of the chi-square negative result.

8.3 Summary of Prompts Used

A separate document, `AI_USAGE.md`, included in the repository root, contains a categorized summary of the types of prompts used during the project. The full conversation history is stored privately in the author's Anthropic account and is available on request.